```
separate (IDL.SEM_PHASE)¶¶¬¬¬¬¬¬------¶¬package body¬¬¬DERIVED¶¬¬¬¬¬¬¬------¶is¶  use DEF_UTIL, VIS_UTIL, MAKE_NOD, REQ_UTIL;¶¶  DERIVED_DECL_LIST¬¬: S
EQ_TYPE;¶¶  function  IS_OPERATION_OF_TYPE¬¬( DECL_ID, TYPE_SPEC : TREE )¬¬return Boolean;¶  function  MAKE_DERIVED_SUBPROGRAM¬¬( DECL_ID¬     : TREE;
¶¬¬¬¬¬  PARENT_TYPE  : TREE;¶¬¬¬¬¬  DERIVED_TYPE : TREE;¶¬¬¬¬¬  H¬     : H_TYPE )¬¬return TREE;¶¶¶¶¬¬¬--========================--¶  function¬¬MAKE_DE
RIVED_SUBPROGRAM_LIST¶¬¬¬--========================--¶¬¬¬¬¬¬( DERIVED_SUBTYPE :¬TREE;¶¬¬¬¬¬¬  PARENT_SUBTYPE  :¬TREE;¶¬¬¬¬¬¬  H¬       :¬H_TYPE¶¬¬¬¬¬
¬)¶¬¬¬¬¬¬return SEQ_TYPE¶  is¶¬      -- RETURNS A LIST OF DERIVED SUBPROGRAMS FOR THE DERIVED TYPE¶    DERIVED_ID¬¬: TREE¬¬:= D( XD_SOURCE_NAME, DERIV
ED_SUBTYPE );¶¶    PARENT_TYPE¬¬: TREE¬¬:= GET_BASE_TYPE( PARENT_SUBTYPE );¶    PARENT_ID¬¬: TREE¬¬:= D( XD_SOURCE_NAME, PARENT_TYPE );¶    PARENT_REG
ION¬¬: TREE¬¬:= D( XD_REGION,¬  PARENT_ID );¶    PARENT_DEF¬¬: TREE¬¬:= GET_DEF_FOR_ID( PARENT_ID );¶¶¶    DECL_LIST¬¬: SEQ_TYPE;¶    DECL¬¬¬: TREE;¶
   DECL_ID¬¬: TREE;¶¶    DERIVED_SUBPROGRAM_LIST¬: SEQ_TYPE¬:= (TREE_NIL, TREE_NIL);¶¶    DERIVED_OF_PARENT_LIST¬: SEQ_TYPE¬:= (TREE_NIL, TREE_NIL);¶
   DERIVED_OF_PARENT¬: TREE;¶    DERIVED_OF_PARENT_SYM¬: TREE;¶    DERIVED_FIRST_KIND_LIST¬: SEQ_TYPE;¶¶    TEMP_FIRST_KIND_LIST¬: SEQ_TYPE;¶    TEMP_
FIRST_KIND¬¬: TREE;¶  begin¶¶¬      -- IF PARENT TYPE IS IN¬VISIBLE PART OF PACKAGE¶¬      -- ... AND CURRENT LOCATION IS NOT IN SAME VISIBLE PART (ER
ROR)¶    if PARENT_REGION.TY = DN_PACKAGE_ID¶    and then DB( XD_IS_IN_SPEC, PARENT_DEF )¶    and then not ( H.IS_IN_SPEC¶¬¬and then PARENT_REGION = D
( XD_SOURCE_NAME, H.REGION_DEF )¶¬¬)¶    then¶¬¬    -- SCAN DECL LIST OF VISIBLE PART IN WHICH PARENT IS DEFINED¶¬¬    -- ... UNTIL PARENT DECLARATION
 IS PASSED¶¬¬    -- ... ALSO, REMEMBER LIST OF DERIVED SUBPROGRAMS OF PARENT¶      DECL_LIST := LIST( D( AS_DECL_S1,¬D( SM_SPEC, PARENT_REGION ) )¬);¶
      loop¶        POP( DECL_LIST, DECL );¶        if DECL.TY = DN_TYPE_DECL or else DECL.TY¬= DN_TASK_DECL then¶¬if D (AS_SOURCE_NAME, DECL) =¬PARENT
_ID¬then¶¬  exit;¶¬end if;¶        end if;¶      end¬loop;¶¶¬¬    -- FOR EACH REMAINING DECLARATION¶      while not IS_EMPTY( DECL_LIST ) loop¶
 POP( DECL_LIST, DECL );¶¶¬¬¬  -- IF IT IS A SUBPROGRAM OR¬ENTRY DECLARATION¶        if DECL.TY = DN_SUBPROG_ENTRY_DECL then¶¶¬¬¬¬¬-- IF IT IS AN OPERA
TION OF THE TYPE (NOTE: ENTRY¬ISN'T)¶¬DECL_ID := D( AS_SOURCE_NAME,¬DECL );¶¬if IS_OPERATION_OF_TYPE( DECL_ID, PARENT_SUBTYPE ) then¶¶¬¬¬¬      -- M
AKE SURE NAME IS IN SYMBOL TABLE¶¬  if D( LX_SYMREP, DECL_ID ).TY = DN_TXTREP then¶¬    D( LX_SYMREP, DECL_ID, STORE_SYM( PRINT_NAME(¬D( LX_SYMREP, DE
CL_ID ) ) ) );¶¬  end if;¶¶¬¬¬¬      -- MAKE NEW¬SUBPROGRAM AND ADD TO LIST¶¬  DERIVED_SUBPROGRAM_LIST := APPEND( DERIVED_SUBPROGRAM_LIST, MAKE_DERI
VED_SUBPROGRAM( DECL_ID, PARENT_SUBTYPE, DERIVED_SUBTYPE, H ) );¶¬end if;¶        end if;¶      end¬loop;¶    end if;¶¶¬      -- REMEMBER LIST OF DERI
VED SUBPROGRAMS OF FIRST KIND¶¬      -- ... (NOTE.¬ DERIVED OF SECOND KIND ARE INSERTED BEFORE IT)¶    DERIVED_FIRST_KIND_LIST := DERIVED_SUBPROGRAM_L
IST;¶¶¬      -- GET LIST OF DERIVED SUBPROGRAMS OF PARENT¶    if PARENT_TYPE.TY in CLASS_DERIVABLE_SPEC¶    and then D( SM_DERIVED, PARENT_TYPE¬) /= T
REE_VOID¶    and then PARENT_TYPE = GET_BASE_STRUCT( PARENT_TYPE )¶    then¶      declare¶        TEMP_DECL_LIST¬: SEQ_TYPE¬:= DERIVED_DECL_LIST;¶
    TEMP_DECL¬¬: TREE;¶      begin¶        while not IS_EMPTY( TEMP_DECL_LIST ) loop¶¬POP( TEMP_DECL_LIST, TEMP_DECL );¶¶¬if D( SM_FIRST, D( AS_SOURCE
_NAME, TEMP_DECL ) ) = PARENT_ID then¶¬  DERIVED_OF_PARENT_LIST := LIST( D( AS_TYPE_DEF,¬TEMP_DECL¬) );¶¬  exit;¶¬end if;¶        end loop;¶      end;
¶    end if;¶¶¬      -- FOR EACH DERIVED SUBPROGRAM OF¬PARENT TYPE¶    while¬not IS_EMPTY( DERIVED_OF_PARENT_LIST ) loop¶      POP( DERIVED_OF_PARENT_
LIST, DERIVED_OF_PARENT );¶¶¬¬    -- MAKE SURE NAME IS IN SYMBOL TABLE¶      DERIVED_OF_PARENT_SYM := D( LX_SYMREP, DERIVED_OF_PARENT );¶      if DERI
VED_OF_PARENT_SYM.TY = DN_TXTREP then¶        DERIVED_OF_PARENT_SYM¬:= STORE_SYM( PRINT_NAME( D( LX_SYMREP,¬DERIVED_OF_PARENT )¬) );¶        D( LX_SYM
REP, DERIVED_OF_PARENT, DERIVED_OF_PARENT_SYM );¶      end¬if;¶¶¬¬    -- FOR EACH DERIVED SUBPROGRAM OF THE FIRST KIND¶      TEMP_FIRST_KIND_LIST :=¬D
ERIVED_FIRST_KIND_LIST;¶      while not IS_EMPTY( TEMP_FIRST_KIND_LIST ) loop¶        TEMP_FIRST_KIND := HEAD( TEMP_FIRST_KIND_LIST );¶¶¬¬¬  -- IF IT
HIDES THE DERIVED SUBPROGRAM¬OF THE SECOND KIND¶        if D( LX_SYMREP, TEMP_FIRST_KIND ) = DERIVED_OF_PARENT_SYM¶        and then ARE_HOMOGRAPH_HEAD
ERS(¬D( SM_SPEC, DERIVED_OF_PARENT¬), D( SM_SPEC, D( SM_DERIVABLE, D( SM_UNIT_DESC, TEMP_FIRST_KIND ) ) ) )¶        then¶¶¬¬¬¬¬-- CAN'T BE DERIVED¬OF S
ECOND¬KIND (LIST NON-EMPTY AT EXIT)¶¬exit;¶        end if;¶¶        TEMP_FIRST_KIND_LIST := TAIL( TEMP_FIRST_KIND_LIST );¶      end¬loop;¶¶¬¬    -- IF
¬NO HIDING¬DERIVED SUBPROGRAM OF FIRST KIND WAS FOUND¶      if IS_EMPTY( TEMP_FIRST_KIND_LIST¬) then¶¶¬¬¬  -- MAKE¬NEW SUBPROGRAM AND ADD TO BEGINNING
 OF LIST¶        DERIVED_SUBPROGRAM_LIST := INSERT( DERIVED_SUBPROGRAM_LIST, MAKE_DERIVED_SUBPROGRAM( DERIVED_OF_PARENT, PARENT_SUBTYPE, DERIVED_SUBTY
PE, H ) );¶      end¬if;¶    end loop;¶¶¬      -- RETURN THE¬LIST OF DERIVED SUBPROGRAMS¶    return DERIVED_SUBPROGRAM_LIST;¶¶  end¬MAKE_DERIVED_SUBPR
OGRAM_LIST;¶¬--==========================--¶¶¶¶¬¬¬--===================--¶  procedure¬¬REMEMBER_DERIVED_DECL¬¬( DECL : TREE )¶¬¬¬--===================--¶  i
s¶    TYPE_DEF : TREE;¶  begin¶    if DECL = TREE_VOID then¶¬¬    -- (INITIALIZATION CALL -- FROM FIXWITH)¶      DERIVED_DECL_LIST := (TREE_NIL, TREE_
NIL);¶      return;¶    end if;¶¶    TYPE_DEF := D( AS_TYPE_DEF, DECL );¶    if TYPE_DEF.TY = DN_DERIVED_DEF and¬then not IS_EMPTY( LIST( TYPE_DEF ) )
 then¶      DERIVED_DECL_LIST := INSERT( DERIVED_DECL_LIST, DECL );¶    end if;¶¶  end¬REMEMBER_DERIVED_DECL;¶¬--==================--¶¶¶¶¬¬¬------------
---------¶  function¬¬IS_OPERATION_OF_TYPE¶¬¬¬---------------------¶¬¬¬¬¬¬¬¬( DECL_ID, TYPE_SPEC : TREE )¶¬¬¬¬¬¬¬¬return Boolean¶  is¶    BASE_TYPE¬: TRE
E¬¬:= GET_BASE_TYPE( TYPE_SPEC );¶    HEADER¬: TREE¬¬:= D( SM_SPEC, DECL_ID );¶    PARAM_CURSOR¬: PARAM_CURSOR_TYPE;¶  begin¶¶¬      -- CHECK FOR ENTR
Y ID; IF SO, IT IS NOT OPERATION¶¬      -- ... (WHILE¬WE'RE AT IT, MAKE SURE IT IS A SUBPROGRAM)¶    if DECL_ID.TY not in CLASS_SUBPROG_NAME then¶
  return False;¶    end if;¶¶¬      -- IF IT IS A¬FUNCTION OR OPERATOR AND RESULT IS OF THE GIVEN TYPE¶    if DECL_ID.TY /= DN_PROCEDURE_ID¶¬¬¬  -- ON
LY¬OTHER POSSIBILITY¶      and¬then GET_BASE_TYPE(¬D( AS_NAME, HEADER ) ) = BASE_TYPE then¶      return True;¶    end if;¶¶¬      -- FOR EACH PARAMETE
R¶    INIT_PARAM_CURSOR( PARAM_CURSOR, LIST( D( AS_PARAM_S, HEADER ) ) );¶    loop¶      ADVANCE_PARAM_CURSOR( PARAM_CURSOR );¶      exit when PARAM_C
URSOR.ID = TREE_VOID;¶¶¬¬    -- IF¬IT IS OF THE PROPER¬TYPE¶      if GET_BASE_TYPE( PARAM_CURSOR.ID¬) = BASE_TYPE then¶¶¬¬¬  -- IT IS AN OPERATION; RE
TURN TRUE¶        return True;¶      end¬if;¶    end loop;¶¶¬      -- NONE FOUND; IT IS NOT AN OPERATION¶    return False;¶¶  end¬IS_OPERATION_OF_TYPE
;¶¬-------------------¶¶¶¶¬¬¬------------------------¶  function¬¬MAKE_DERIVED_SUBPROGRAM¶¬¬¬------------------------¶¬¬¬¬¬¬¬¬( DECL_ID¬     : TREE;¶¬¬¬
¬¬¬¬  PARENT_TYPE  : TREE;¶¬¬¬¬¬¬¬¬  DERIVED_TYPE : TREE;¶¬¬¬¬¬¬¬¬  H : H_TYPE¶¬¬¬¬¬¬¬¬)¶¬¬¬¬¬¬¬¬return TREE¶  is¶    NEW_ID¬: TREE¬¬:= COPY_NODE( DECL_ID

 );¶    NEW_DEF¬: TREE¬¬:= MAKE_DEF_FOR_ID(¬NEW_ID, H¬);¶    HEADER¬: TREE¬¬:= D( SM_SPEC, DECL_ID );¶    NEW_HEADER¬: TREE¬¬:= COPY_NODE( HEADER );¶
   NEW_TYPE_MARK¬: TREE¬¬:= MAKE_USED_NAME_ID(¬LX_SYMREP¬=> D( LX_SYMREP,¶¬¬¬¬¬¬¬¬D( XD_SOURCE_NAME, DERIVED_TYPE ) ),¶¬¬¬¬¬¬¬¬SM_DEFN => D( XD_SOURCE_N
AME,¬DERIVED_TYPE )¶¬¬¬¬¬¬¬);¶    NEW_PARAM_LIST¬: SEQ_TYPE¬:= (TREE_NIL, TREE_NIL);¶  begin¶    D( SM_FIRST,¬ NEW_ID, NEW_ID );¶    D( SM_UNIT_DESC, N
EW_ID, MAKE_DERIVED_SUBPROG( SM_DERIVABLE => DECL_ID ) );¶    D( XD_STUB,¬ NEW_ID, TREE_VOID¬);¶    D( XD_BODY,¬ NEW_ID, TREE_VOID¬);¶    D( SM_SPEC,¬
 NEW_ID, NEW_HEADER¬);¶    D( SM_ADDRESS,¬ NEW_ID, TREE_VOID¬);¶¶    MAKE_DEF_VISIBLE( NEW_DEF, NEW_HEADER );¶    if HEADER.TY = DN_FUNCTION_SPEC and¬
then GET_BASE_TYPE(¬D( AS_NAME, HEADER ) ) = PARENT_TYPE then¶      D( AS_NAME, NEW_HEADER,¬NEW_TYPE_MARK );¶    end if;¶¶    declare¶      PARAM_CURS
OR¬: PARAM_CURSOR_TYPE;¶    begin¶      INIT_PARAM_CURSOR( PARAM_CURSOR, LIST( D( AS_PARAM_S,¬HEADER ) ) );¶      loop¶        ADVANCE_PARAM_CURSOR(¬P
ARAM_CURSOR );¶        exit when PARAM_CURSOR.ID = TREE_VOID;¶        declare¶¬NEW_PARAM_ID¬: TREE¬:= COPY_NODE( PARAM_CURSOR.ID¬);¶¬NEW_PARAM_DECL¬:
TREE¬:= COPY_NODE( PARAM_CURSOR.PARAM );¶        begin¶        D( SM_FIRST,  NEW_PARAM_ID, NEW_PARAM_ID );¶        D( XD_REGION, NEW_PARAM_ID, NEW_ID
);¶        D( AS_SOURCE_NAME_S, NEW_PARAM_DECL, MAKE_SOURCE_NAME_S( LIST¬=> SINGLETON( NEW_PARAM_ID ) ) );¶        if GET_BASE_TYPE( PARAM_CURSOR.ID )
 = PARENT_TYPE then¶¬D( SM_OBJ_TYPE, NEW_PARAM_ID,¬DERIVED_TYPE );¶¬if D( SM_INIT_EXP, PARAM_CURSOR.ID ) /=¬TREE_VOID¬then¶¬  D( SM_INIT_EXP, NEW_PARA
M_ID, MAKE_CONVERSION( AS_NAME =>¬NEW_TYPE_MARK,¶¬¬¬AS_EXP =>¬D (SM_INIT_EXP, PARAM_CURSOR.ID), SM_EXP_TYPE => DERIVED_TYPE));¶¬end if;¶        end if
;¶        NEW_PARAM_LIST := APPEND( NEW_PARAM_LIST,¬NEW_PARAM_DECL );¶        end;¶      end¬loop;¶      D( AS_PARAM_S, NEW_HEADER, MAKE_PARAM_S( LIST
 => NEW_PARAM_LIST¬) );¶    end;¶¶¬      -- ALSO DERIVE INEQUALITY IF THIS¬IS EQUALITY OPERATOR¶    if NEW_ID.TY = DN_OPERATOR_ID then¶      declare¶
      UNEQUAL_ID¬: TREE¬:= D( XD_NOT_EQUAL,¬NEW_ID );¶      begin¶        if UNEQUAL_ID /= TREE_VOID then¶¬UNEQUAL_ID := COPY_NODE( UNEQUAL_ID );¶¬D(
 SM_FIRST, UNEQUAL_ID, UNEQUAL_ID );¶¬D( SM_SPEC,  UNEQUAL_ID, NEW_HEADER );¶¬if D( LX_SYMREP, UNEQUAL_ID ).TY = DN_SYMBOL_REP then¶¬  MAKE_DEF_VISIBL
E(¬MAKE_DEF_FOR_ID( UNEQUAL_ID, H ), NEW_HEADER );¶¬end if;¶¬D( SM_UNIT_DESC, UNEQUAL_ID, MAKE_DERIVED_SUBPROG( SM_DERIVABLE => D(¬XD_NOT_EQUAL, NEW_I
D ) ) );¶¬D( XD_NOT_EQUAL, NEW_ID,     UNEQUAL_ID¬);¶        end if;¶      end;¶    end if;¶¶    return NEW_ID;¶  end¬MAKE_DERIVED_SUBPROGRAM;¶¬------
-----------------¶¶¶¬--------¶end¬DERIVED;¶¬--------¶
```